

Advanced SQL Functionality 1

Overview

9 Sept 2026

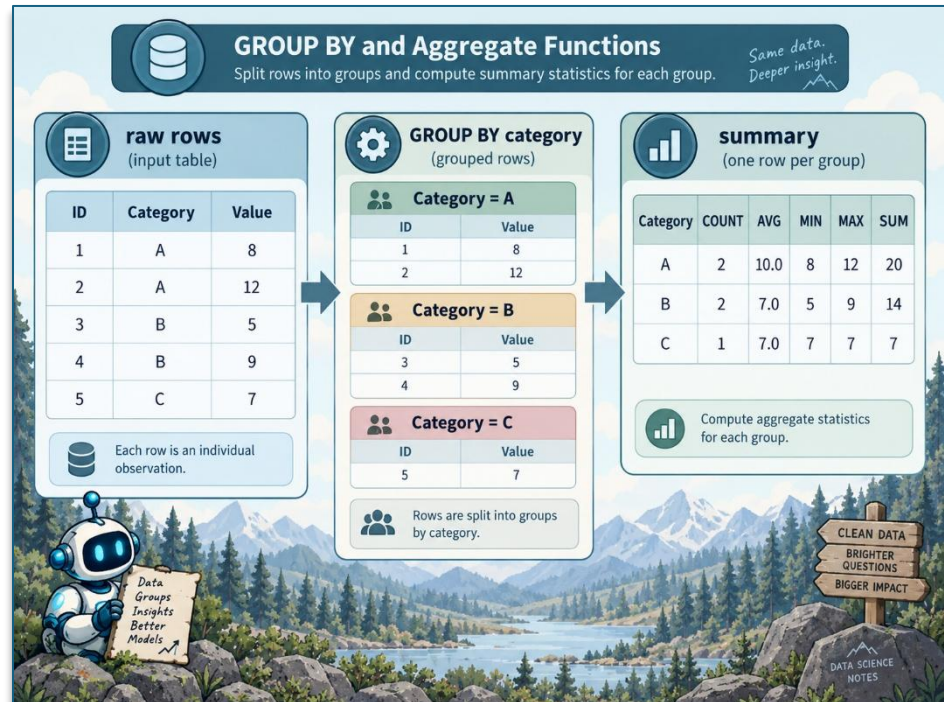
- apply advanced SQL functions
- practice the basics of regular expressions

Async

Subqueries and Aggregate Functions

Aggregate Functions With Group By

Aggregate functions summarize values across rows, and **GROUP BY** allows the same calculation to be performed separately for defined groups.



ChatGPT 5.6 Sol

- aggregate functions can calculate values for an entire table or for groups within a table
- common aggregate functions:
 - `COUNT()`
 - `MIN()`
 - `MAX()`

	○ <i>SUM()</i> ○ <i>AVG()</i> GROUP BY: • divides rows into groups based on one or more specified columns • the aggregate function is then calculated independently for each group • example use case: ○ count patients separately by gender rather than running a separate COUNT query for every gender
Labeling Grouped Results	Grouped aggregate results are easier to interpret when the grouping column is included in the <i>SELECT</i> clause. • selecting only <i>COUNT()</i>: ○ returns the counts without identifying which group produced each value • include the column used for grouping in <i>SELECT</i> to label each result Multiple columns can be included in <i>GROUP BY</i>. • grouping by gender and race produces one result for each existing gender-race combination <i>SELECT</i> And <i>GROUP BY</i> Requirements. • when aggregate functions are combined with ordinary columns: ○ the non-aggregate columns in <i>SELECT</i> must also appear in <i>GROUP BY</i>

	<i>GROUP BY</i> is not required when <i>SELECT</i> contains only aggregate functions. - if <i>SELECT</i> contains an aggregate function and another column: - that additional column must also appear in <i>GROUP BY</i> A common aggregate-query error is forgetting to match non-aggregate <i>SELECT</i> columns with the <i>GROUP BY</i> clause,
Distinct Values	<i>DISTINCT</i> allows aggregate functions to operate on unique values rather than simply counting every row. <i>COUNT(*)</i>: - counts rows regardless of duplicated values <i>COUNT(DISTINCT column)</i>: - counts the number of unique values in the specified column <i>DISTINCT</i> is useful when the question concerns unique entities or values rather than total records.
Limitations Of Grouping	<i>GROUP BY</i> can become restrictive when the desired output includes a value associated with an aggregate result. - example: - finding a patient's most recent weight <i>MAX(weight_date)</i>: - can identify the most recent measurement date

	• Including patient_id is possible by grouping on patient_id • including the corresponding weight value: ○ creates a problem because weight would also need to be added to <i>GROUP BY</i> • grouping by weight: ○ changes the grouping structure and no longer answers the original question The requirement is not merely to find the maximum date but to retrieve the row associated with that date.
Subqueries	A subquery, also called a nested query, places one SQL query inside another query. • the result of a SQL query: ○ a table ○ or relation • query results: ○ usually temporary ○ exist only while the query is being executed A temporary query result can still be used as a data source within another query. • the inner query: ○ enclosed in parentheses • the subquery result: ○ assigned a name or alias ○ allows the outer query to reference it like another table • once named

- the subquery result can participate in operations such as joins

Most Recent Weight Example

The solution separates the problem into two logical stages.

- inner query:
 - calculate *MAX(weight_date)* for each patient
 - group the calculation by patient_id
 - return patient_id and the corresponding maximum date
 - assign the resulting temporary table a name such as patient_max_date

outer
query

```
SELECT
    PATIENT_ID,
    WEIGHT,
    WEIGHT_DATE
FROM
    PatientWt wt,
    (SELECT
        PATIENT_ID,
        MAX (WEIGHT_DATE) AS MaxDt
    FROM
        PatientWt
    GROUP BY
        PATIENT_ID) PatMaxDt
WHERE
    wt.PATIENT_ID = PatMaxDt.PATIENT_ID
    AND wt.WEIGHT_DATE = PatMaxDt.MaxDt
```

inner
query

	• outer query: ○ use the original patient-weight table together with patient_max_date ○ join the temporary result back to the original table ○ retrieve the weight value associated with the maximum date
Join Conditions	The outer query must match the subquery result to the correct row in the original table. • join on patient_id to ensure the result belongs to the correct patient • join on weight_date and the calculated maximum date to identify the patient's most recent measurement row • both conditions are necessary because many patients and many measurement dates may exist in the original table
Why The Approach Works	The aggregation is completed inside the subquery before the outer query retrieves the remaining values. • the subquery: ○ reduces each patient's measurements to the relevant maximum date • the outer query: ○ no longer needs to perform the aggregation ○ can therefore select patient_id, weight_date, and weight without adding weight to GROUP BY

	This allows the query to return values from the specific row identified by the aggregate calculation.
Subquery Tradeoff	Subqueries provide a powerful way to break complex SQL problems into intermediate results, but deeply nested queries can become difficult to read. • multiple nested subqueries: ○ can produce many layers of parentheses and temporary results • later techniques: ○ can provide cleaner ways to organize queries containing many intermediate calculations
Key Ideas	Aggregate functions summarize sets of rows. • <i>GROUP BY</i> ○ determines the groups over which aggregate functions operate • subqueries ○ allow the result of one query to become an intermediate relation used by another query ○ especially useful when an aggregate identifies a row and additional values from that same row must be retrieved

Set Operations	
Boolean Logic Refresher	Boolean logic determines whether each tuple satisfies the conditions in a query. AND: • true only when both conditions are true OR: • true when either condition or both conditions are true NOT: • reverses the result of a condition During selection, each tuple is evaluated and retained only when the condition evaluates to true.
Core Set Operations	Set operations combine the results of multiple relational queries. UNION : • combines all tuples from both result sets INTERSECT: • keeps only tuples that appear in both result sets EXCEPT or MINUS: • keeps tuples from the first result set that do not appear in the second These operations correspond closely to Boolean <i>OR</i>, <i>AND</i>, and <i>NOT</i> logic,

Relational Algebra Connection	Relational algebra provides the mathematical foundation for the relational model and can represent the same operations expressed in SQL. • projection operator: ○ represents selecting attributes or columns • selection conditions: ○ correspond to filtering performed by a <i>WHERE</i> clause • relation inside the expression: ○ represents the source table Set operators can then combine the results of complete relational expressions.
UNION	<i>UNION</i> is used when rows satisfying either of two query conditions should appear in the final result. • example question: ○ find patients who are age 18 or older or have an overdue billing status First query: • select the desired patient attributes from the patient table where age is at least 18 Second query: • select the same attributes from the billing table where billing status is overdue

	<i>UNION:</i> • combines the two result sets into one final relation • a tuple is retained if it appears in: ○ the first result ○ the second result ○ both More than two query results can be combined with <i>UNION</i>.
UNION And Duplicate Rows	<i>UNION</i> removes duplicate tuples automatically. • if the same tuple appears in both result sets: ○ it appears only once in the <i>UNION</i> result <i>UNION ALL:</i> • preserves duplicate tuples Use <i>UNION ALL</i> when duplicate rows are meaningful and should remain in the result.
Union Compatibility	Queries must be union compatible before their results can be combined. • each query must return the same number of columns • corresponding columns must have compatible data types • column names do not have to be identical • the position and type of the returned attributes must align Mismatched column counts or incompatible types are common causes of <i>UNION</i> errors.

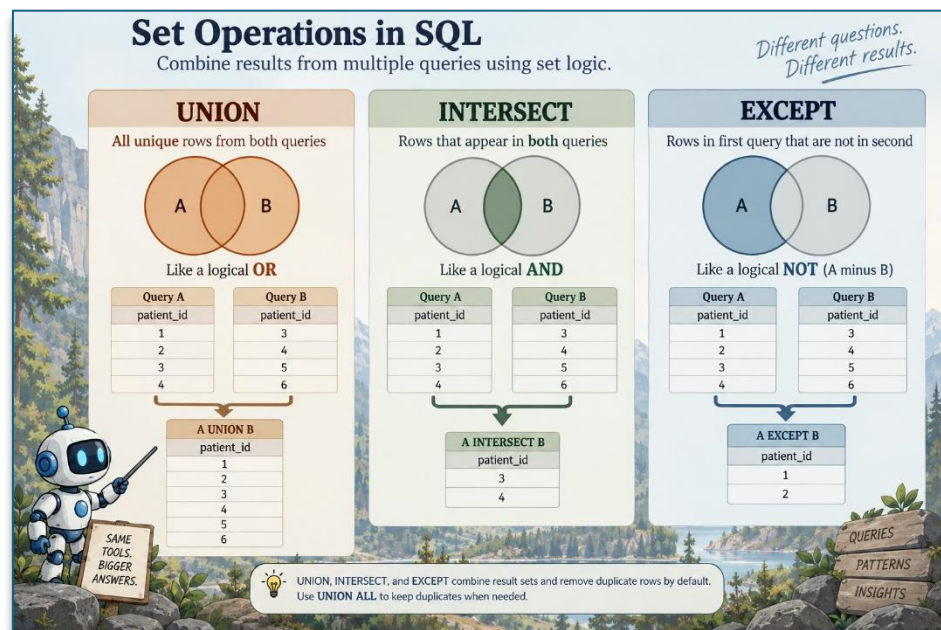
INTERSECT	<i>INTERSECT</i> returns only tuples that appear in both query results. • example question: ○ find patients who are age 18 or older and also have an overdue billing status First query: • identifies patients meeting the age condition Second query: • identifies patients meeting the billing condition <i>INTERSECT</i>: • retains only patients present in both result sets <i>INTERSECT</i> therefore corresponds to Boolean <i>AND</i> logic.
EXCEPT And MINUS	<i>EXCEPT</i> or <i>MINUS</i> returns tuples from the first result set that are not present in the second result set. • example question: ○ find patients who are age 18 or older and do not have an overdue billing status First query: • identifies patients meeting the age condition Second query: • identifies patients with overdue bills

	EXCEPT: • removes tuples from the first result that also appear in the second • rows that exist only in the second result do not affect the final output The order of the result sets matters because subtraction is directional.
Set Operations In SQL	Set operations can be placed directly between compatible SQL queries. • general pattern: ○ first <i>SELECT</i> query ○ <i>UNION, INTERSECT, or EXCEPT</i> ○ second <i>SELECT</i> query The operator determines how the two result sets are combined.
Set Operations Versus Joins	Set operations and joins combine relational data in different ways. • joins: ○ primarily combine columns from different tables • set operations: ○ primarily combine rows from different query results • <i>INTERSECT</i>: ○ can often express logic similar to an <i>INNER JOIN</i> • <i>EXCEPT</i> or <i>MINUS</i>:

- can often be represented using an anti-join
- *UNION*:
 - has no equivalent join operation because its purpose is to append compatible rows from separate result sets

Key Relationships

- *UNION* corresponds to *OR*
- *INTERSECT* corresponds to *AND*
- *EXCEPT* or *MINUS* corresponds to *NOT* or set subtraction
- *UNION* combines rows from either result
- *INTERSECT* keeps rows common to both results
- *EXCEPT* keeps rows from the first result that are absent from the second



Set operations require compatible result structures and operate on complete query result sets.

CASE Statements	
Purpose Of CASE Statements	CASE statements allow SQL to transform existing values into derived categories. • continuous or highly specific values can be converted into broader categorical variables ○ individual ages can be grouped into ranges ▪ for instance: • 0 – 18 • 19 – 35 • 36 – 50 • common applications include analyzing age groups associated with: ○ clinic visits ○ diseases ○ procedures ○ other outcomes Categorical age groups can make analysis easier when the goal is to compare populations rather than individual ages. Step 1: Calculate Age • before creating age categories: ○ the patient's age must first be calculated from the date of birth ○ age is derived by comparing the person's birth date with the current date ○ the exact calculation syntax depends on the database management system

- date and datetime values may require conversion so that compatible data types are used in the calculation
- the resulting age:
 - should represent completed years rather than conventional numerical rounding
 - a person remains age 64 until their 65th birthday even when their calculated age is 64.9
 - the lecture uses *FLOOR()* to remove the decimal portion and round the calculated value downward
- the calculated field can be assigned an alias such as age

Step 2: Examine The Age Distribution

- category boundaries should be informed by the underlying data
 - calculate age for the full population
- ***COUNT()***:
 - the number of people at each age
- ***GROUP BY:***
 - the calculated age so each age receives its own count
- ***ORDER BY:***
 - age to make the distribution easier to inspect
- review the distribution before deciding where meaningful category boundaries should be placed

- real datasets may contain:
 - peaks
 - gaps
 - uneven concentrations across different ages

Step 3: Define Categories With *CASE*

- *CASE* statement:
 - evaluates conditions in sequence
 - returns a specified value when a condition is satisfied
- general structure:

```
CASE
  WHEN condition THEN result
  WHEN condition THEN result
  ELSE result
END
```

WHEN defines a condition to test

THEN specifies the value assigned when that condition is true

additional

WHEN clauses define additional categories

ELSE provides a value for records that do not satisfy any previous condition

END closes the *CASE* statement

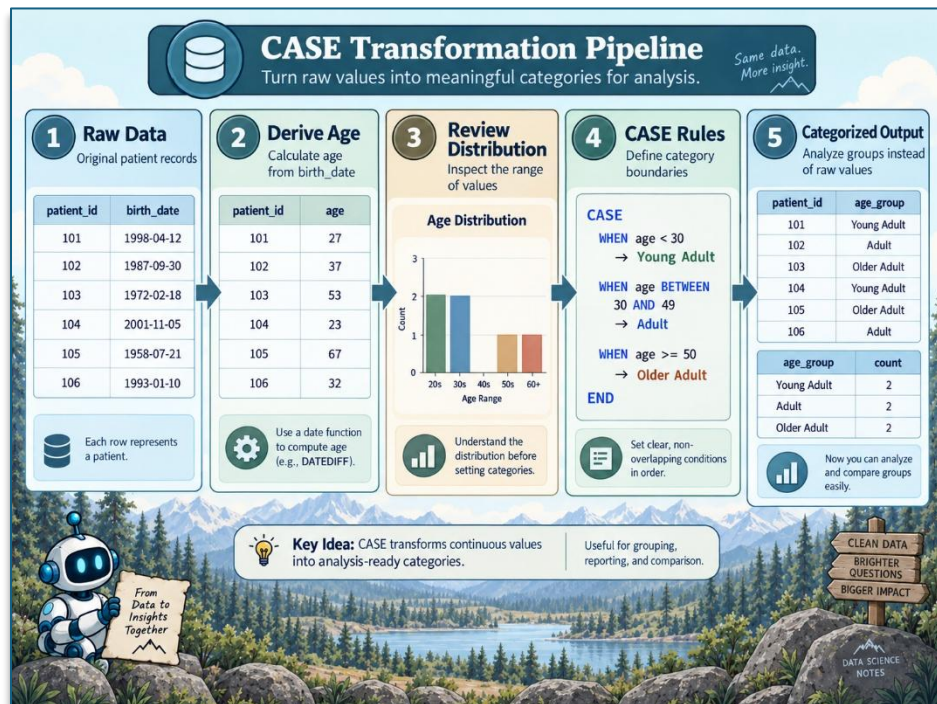
- the complete *CASE* expression can be assigned an alias such as `age_group`

	• Why <i>ELSE</i> Matters ○ <i>ELSE</i> acts as the catch-all condition in a <i>CASE</i> statement ○ records that do not satisfy any <i>WHEN</i> condition still need a valid outcome ○ without an appropriate catch-all category, some values may not receive the intended classification Step 4: Count Records Within Each Category • once the categorical variable has been created ○ aggregate functions can summarize the new groups • when individual records are no longer needed ○ remove <code>person_id</code> • use <i>COUNT(*)</i> ○ to calculate the number of people assigned to each age group • retain the <i>CASE</i> expression ○ it defines the categorical variable being analyzed • <i>GROUP BY</i> the age group ○ ensures one count is returned for each category The result provides a frequency distribution across the newly created categorical variable.
Grouping By A SELECT Position	SQL can sometimes reference expressions by their position in the <i>SELECT</i> list. • <i>GROUP BY 1</i> ○ group by the first expression in <i>SELECT</i>

- this can avoid repeating a long *CASE* expression inside the *GROUP BY* clause
- **ORDER BY**
 - can use the same positional shorthand
- **GROUP BY 1**
 - is especially convenient when the first selected value is a complex calculated expression
- **positional references**
 - improve brevity but require awareness of the *SELECT* column order

Overall Transformation

The workflow converts raw data into an analysis-ready categorical variable.



	begin with a continuous or detailed source value • derive the value needed for analysis, such as age • inspect its distribution • define meaningful category boundaries • use <i>CASE</i> to assign each record to a category • use <i>GROUP BY</i> and <i>COUNT()</i> to summarize the categories The result is a categorical representation that can be used for further analysis.
Key Ideas	• <i>CASE</i> provides conditional logic within a SQL query • <i>CASE</i> can transform continuous or detailed data into categorical variables • <i>WHEN</i> and <i>THEN</i> define individual classification rules • <i>ELSE</i> handles values that do not match earlier conditions • <i>END</i> completes the <i>CASE</i> expression • <i>category design</i> should be informed by the distribution and purpose of the data • <i>aggregate functions</i> can summarize records after <i>CASE</i> creates the categories • <i>GROUP BY 1</i> can provide shorthand for grouping by a complex first <i>SELECT</i> expression
CASTing	Sometimes you want to work with data in a format other than its native format. • <i>CAST</i> ○ convert from one datatype to another on the fly • example:

Field1	Field2	Field3
varchar	integer	numeric
123	987	5.6523
456	654	9.6701
789	321	10.8711

- the concatenate function is not designed to work with numeric data types

- Field1 is varchar datatype
- Field2 is an integer

```
--this query would error out
SELECT Field1 || Field2 FROM TABLE;

--this query would run successfully
SELECT Field1 || CAST(Field2 AS VARCHAR) FROM TABLE;
```

- casting may also have unintended consequences, so be cautious
- Field3
 - numeric datatype
 - four digits to the right of the decimal
- What would happen if I CAST Field3 as an integer, which by definition has no floating point?

```
--casting Field3 to an integer
SELECT CAST
  (Field3 AS INTEGER) AS Field3_int
FROM TABLE
```

Field3_int

integer

5

9

10

- note that the digits after the decimals have dropped off after the cast is performed
- so, if you perform a cast like this, just make sure that this is the behavior you want!

There is a shortcut for the cast function--these two statements are equivalent:

```
--use two colons to replace the full cast syntax
SELECT CAST
    (Field3 AS INTEGER) AS Field3_int
FROM TABLE ;

SELECT
    Field3::INTEGER AS Field3_int
FROM TABLE ;
```

- Note: you cannot cast whatever you want
 - `cast('ABC' as integer)` will always result in an error

MariaDB Note

The lecture demonstrates a PostgreSQL-specific method for calculating age, so that portion should not be copied directly into MariaDB.

- in MariaDB, completed age in years can be calculated directly with `TIMESTAMPDIFF()`
 - `TIMESTAMPDIFF(YEAR, birth_datetime, CURDATE())`
- this already returns the number of completed year boundaries

	the FLOOR() and division-by-365.25 approach from the PostgreSQL demonstration is unnecessary for the MariaDB version
Fun With Dates	
Why Dates Are Complicated	Dates can be difficult to work with because the same date can be represented in many different formats. - examples: - June 1, 2023 2023-01-06 6/1/2023 1 June 2023 - different relational database management systems may use different default date formats - common default format - YYYY-MM-DD - Oracle commonly uses - DD-Mon-YY Differences in date formatting can create interoperability problems when data moves between systems.
Importing Data With Dates	Date values may need to be transformed or cleaned before they can be imported successfully. - if the source date format matches the target database format: - choose the appropriate date datatype - import the data directly

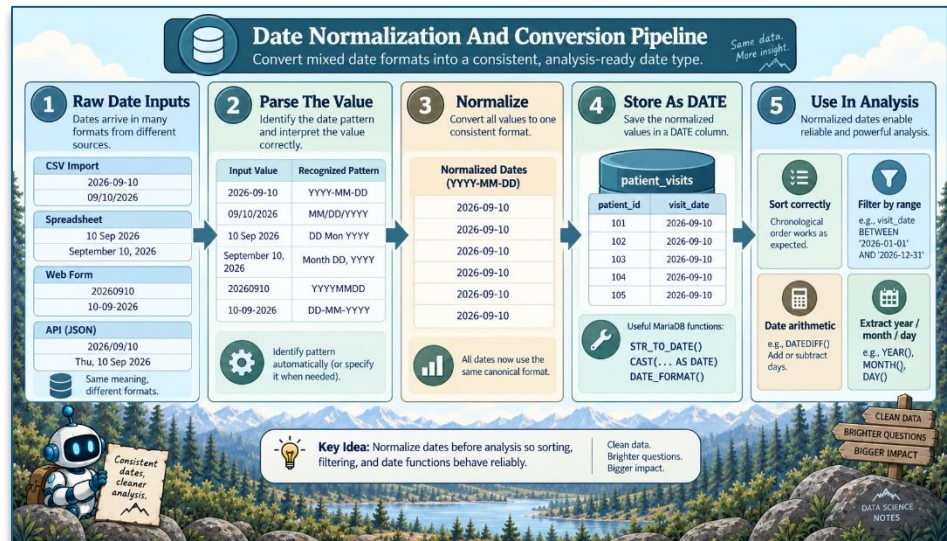
	• if the source and target formats do not match: ○ transform the dates before import ○ transform the dates with a function during import Import the dates as strings, convert them after loading, then remove the original string values.
Choosing A Date Datatype	The target column should use a datatype appropriate for the information being stored. • PostgreSQL date-related datatypes include: ○ timestamp with time zone ○ timestamp without time zone ○ date ○ time with time zone ○ time without time zone ○ interval The correct datatype depends on whether the data represents a date, time, datetime, time zone, or duration.
Converting Date Formats	Date conversion is often necessary when source data uses a different format from the database. • example source format: ○ MM/DD/YYYY • PostgreSQL provides TO_DATE() ○ formats text into a date value ○ general form:

- `to_date(<date field>, <formatting hint>)`

- example:

- `to_date(birth_date, 'MM/DD/YYYY')`

- the formatting hint tells PostgreSQL how the source string is structured



ChatGPT 5.6 Sol

The exact conversion function and syntax depend on the RDBMS.

Using CAST For Date Conversion

CAST may be sufficient when the source value can be converted directly into the desired date datatype.

- CAST can be simpler than a formatting function when the source data is already close to a recognized database format
- messier or nonstandard date strings may require a dedicated date-formatting function
- the best conversion method depends on the source data and database dialect

Date Functions	Once dates are stored correctly, date functions can be used to manipulate and analyze them. • date-function syntax differs substantially across database systems • the course examples use PostgreSQL syntax • similar operations in another RDBMS may require different functions or syntax
Common Date Operations	SQL date functions can support several types of operations. • date arithmetic: ○ add units of time to a date or time ○ subtract units of time from a date or time • date differences: ○ calculate the distance between two dates or times • current date and time: ○ retrieve the current date or datetime from the database system • date-part extraction: ○ extract an individual component such as year, month, or day ○ example: ▪ extract the year from 2023-06-01

Key Ideas	• date formatting varies across source data and database systems • format mismatches commonly require conversion during data import or transfer • the appropriate date datatype should be selected before loading data • dates can be transformed before import, during import, or after temporary string-based loading • PostgreSQL provides <i>TO_DATE()</i> for parsing formatted date strings • <i>CAST</i> may work when values are already compatible with the target datatype • date functions support arithmetic, differences, current datetime values, and extraction of date components • date syntax is highly database-specific, so functions often need to be adapted when changing RDBMSs
Basic String Functions	
Purpose Of String Functions	String functions allow SQL to inspect, combine, extract, and transform text stored in database fields. • common uses include: ○ combining text from multiple values ○ measuring text length ○ locating patterns within text ○ extracting portions of strings ○ retrieving characters from either end of a string ○ replacing text patterns

	- the lecture uses a table containing sections of the story Peter and the Wolf to demonstrate these operations - each row contains an identifier, an index describing the section, and a text field containing part of the story
Concatenation	Concatenation combines two or more strings into a single string. - in PostgreSQL, the concatenation operator is <code> </code> - multiple text values can be joined together in sequence - complete <i>SELECT</i> queries can also be placed inside parentheses and concatenated after their results are returned - the outer <i>SELECT</i> evaluates the completed concatenation expression - a <i>FROM</i> clause is not required when the outer <i>SELECT</i> is operating only on values or completed expressions rather than directly reading another table - example concept: - first text value <code>//</code> second text value - concatenated values are placed directly next to one another - spaces or other separators must be added explicitly when needed - example concept: - first text value <code>// ' '</code> second text value - the literal space becomes part of the resulting string

LENGTH()	<i>LENGTH()</i> measures the length of a string. • general form: ◦ <i>LENGTH(text)</i> • the function can be applied to a text column for every row in a table • the result can be assigned an alias such as text_length • text length can be useful for identifying unusually long or short values • the result can also support data-quality checks and later filtering or analysis
POSITION()	<i>POSITION()</i> identifies where a specified pattern first occurs within a string. • general PostgreSQL form: ◦ <i>POSITION('Peter' IN text)</i> • the function searches the text field for the requested pattern • the returned value identifies the starting position of the first match • only the first occurrence is returned when the pattern appears multiple times • a result of 0 indicates that the pattern was not found • <i>POSITION()</i> can therefore be used both to locate text and to determine whether a pattern exists at all
SUBSTRING()	<i>SUBSTRING()</i> extracts part of a larger string. • the function requires: ◦ the source string

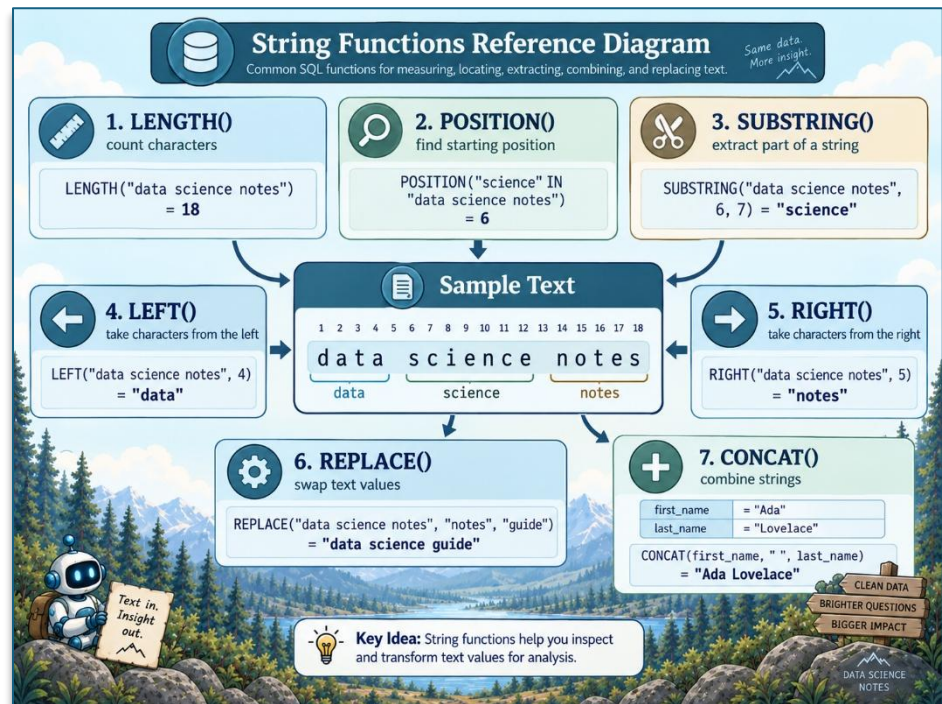
	○ the starting position ○ the number of characters to retrieve • the starting position can be calculated dynamically with another function • example workflow: ○ use <i>POSITION()</i> to find the first occurrence of Peter ○ use that position as the starting point for <i>SUBSTRING()</i> ○ return a fixed number of characters beginning at that location Functions can therefore be nested so that the result of one function becomes an argument to another.
Combining <i>POSITION()</i> And <i>SUBSTRING()</i>	<i>POSITION()</i> and <i>SUBSTRING()</i> can work together to extract text surrounding a known pattern. • first locate the target pattern • use the returned position as the starting point for extraction • specify how many characters should be returned ○ this allows extraction to begin at a location that may differ from row to row • special handling is needed when the search pattern does not exist • if <i>POSITION()</i> returns 0: ○ <i>SUBSTRING()</i> may still attempt to extract characters from the beginning of the string • filtering out rows where the position equals 0 prevents misleading results

	• example condition: ◦ <i>WHERE POSITION('Peter' IN text) <> 0</i>
LEFT() And RIGHT()	<i>LEFT()</i> and <i>RIGHT()</i> extract a fixed number of characters from either end of a string. • general forms: ◦ <i>LEFT(text, 12)</i> ◦ <i>RIGHT(text, 12)</i> • <i>LEFT()</i>: ◦ returns the specified number of characters beginning at the left side of the string • <i>RIGHT()</i>: ◦ returns the specified number of characters beginning at the right side of the string These functions are useful when relevant information consistently appears at a known end of a text value.
REPLACE()	<i>REPLACE()</i> substitutes one text pattern for another. • general form: ◦ <i>REPLACE(text, 'Peter', 'Emily')</i> • first argument: ◦ identifies the source string • second argument:

	○ identifies the text to find • third argument: ○ specifies the replacement text • all matching occurrences within the processed string are replaced • <i>REPLACE()</i>: ○ provides functionality similar to search-and-replace in a text editor ○ it can be used for: ○ cleaning ○ standardizing ○ transforming stored text
Nested String Operations	String functions can be combined to perform more complex transformations. • one function can generate an input value for another function • <i>POSITION()</i>: ○ can determine where <i>SUBSTRING()</i> should begin • concatenation: ○ can combine values returned from multiple expressions • filtering conditions: ○ can remove rows where a string operation would not produce a meaningful result

- complex string manipulation:
 - often built by combining several simple functions rather than relying on one large operation

Key Functions



ChatGPT 5.6 Sol

concatenation:

- combine multiple strings into one value

LENGTH():

- measure string length

POSITION():

- find the first occurrence of a pattern

SUBSTRING():

- extract a specified portion of a string

	<i>LEFT()</i>: • extract characters from the beginning of a string <i>RIGHT()</i>: • extract characters from the end of a string <i>REPLACE()</i>: • substitute one text pattern for another
Key Ideas	SQL provides functions for both examining and transforming text data. • string functions can be used individually or nested together • literal spaces and separators must be explicitly included when concatenating values • <i>POSITION()</i> returns only the first matching location • missing search patterns should be handled before using their position in another string operation • string manipulation is useful for data cleaning, transformation, validation, and feature preparation
MariaDB Adaptation	The lecture demonstrates PostgreSQL syntax, but most of these operations have direct MariaDB equivalents. • MariaDB normally uses <i>CONCAT()</i> rather than <code> </code> for portable string concatenation • example: ◦ <i>CONCAT(first_text, ' ', second_text)</i> • <i>POSITION()</i> is available in MariaDB with similar syntax

	• example: ◦ <i>POSITION('Peter' IN text)</i> • <i>LOCATE()</i> provides another common MariaDB form • example: ◦ <i>LOCATE('Peter', text)</i> • <i>SUBSTRING()</i>, <i>LEFT()</i>, <i>RIGHT()</i>, and <i>REPLACE()</i> are also available in MariaDB • MariaDB <i>LENGTH()</i> returns the string length in bytes rather than necessarily the number of characters • <i>CHAR_LENGTH()</i> should be used when the desired measurement is the number of characters • example: ◦ <i>CHAR_LENGTH(text)</i>
LIKE	
Purpose Of LIKE	LIKE is used when a query needs to match a text pattern instead of an exact value. • LIKE ◦ useful for filtering messy or inconsistent text data ◦ can also be used for fuzzy joins when two values are similar enough to match by pattern • common use cases include: ◦ synonyms ◦ partial names

	○ inconsistent descriptions ○ text values where the target word may appear at the beginning, middle, or end
Filtering With LIKE	<i>LIKE</i> can be used in the <i>WHERE</i> clause to keep rows that match a pattern. • example problem: ○ multiple lab test names may refer to a COVID test • example values: ○ SARS-CoV-2 PCR ○ COVID-19 and flu panel ○ SARS-CoV-2 antigen ○ Influenza panel • the first three values refer to COVID-related testing • they do not use the same exact text • if COVID tests are known to include either COVID or SARS-CoV-2, <i>LIKE</i> can match both patterns • example:<pre>SELECT * FROM patient_labs WHERE lab_test_name LIKE '%SARS-CoV-2%' OR lab_test_name LIKE '%COVID%'</pre> • this returns rows where the lab test name contains either SARS-CoV-2 or COVID

Wildcards

Wildcards define how flexible the LIKE pattern should be.

- % means any sequence of zero or more characters
- _ means exactly one character
- % is useful when the target pattern may appear anywhere in the value
- example:
 - '%COVID%' matches COVID at the beginning, middle, or end of the text

LIKE and Wildcards
Use the LIKE operator to find text that matches a pattern.

*Small symbols.
Big possibilities.*

Common Wildcards	
Symbol	Meaning
%	matches zero or more characters
_	matches exactly one character
[]	matches any one character in the list
[^]	matches any one character not in the list

Example Data	
customer_id	name
1	Alice Johnson
2	Alan Smith
3	Alicia Brown
4	Bob Williams
5	Robert Jones
6	Carol White
7	Charles King
8	Alina Garcia
9	Billy Adams
10	Albert Lee

Pattern Examples		
Pattern	Matches	Description
A%	Alice, Alan, Alicia, Alina, Albert	starts with 'A'
%son	Johnson	ends with 'son'
%li%	Alice, Alicia, Alina	contains 'li'
A_%	Alan, Alice, Alina, Albert	'A' followed by any one character
B[bo]%	Bob, Billy	'B' followed by 'b' or 'o'
C[^a]%	Charles	starts with 'C' but not 'Ca'
%%	all rows	matches anything

Query Examples *Same data. Different questions.*

Query	Results
SELECT * FROM customers WHERE name LIKE 'A%'	Alice, Alan, Alicia, Alina, Albert
SELECT * FROM customers WHERE name LIKE '%son'	Johnson
SELECT * FROM customers WHERE name LIKE '%li%'	Alice, Alicia, Alina
SELECT * FROM customers WHERE name LIKE 'A_%'	Alan, Alice, Alina, Albert
SELECT * FROM customers WHERE name LIKE 'B[bo]%	Bob, Billy
SELECT * FROM customers WHERE name LIKE 'C[^a]%	Charles

PATTERNS TURN DATA INTO ANSWERS.

QUERIES
PATTERNS
INSIGHTS

LIKE is great for simple pattern matching. For more complex patterns, use regular expressions (regex).

EXPLORE
PRACTICE
APPLY

ChatGPT 5.6 Sol

A pattern without wildcards is much closer to an exact match.

Handling Case Sensitivity

LIKE matching can be affected by differences in capitalization depending on the database and settings.

- messy data may contain inconsistent case
- example variations:
 - COVID
 - Covid

- covid

- case can be standardized during comparison by applying `LOWER()` or `UPPER()`

- lowercase example:

```
SELECT *  
FROM patient_labs  
WHERE LOWER(lab_test_name) LIKE '%sars-cov-2%'  
OR LOWER(lab_test_name) LIKE '%covid%'
```

- uppercase example:

```
SELECT *  
FROM patient_labs  
WHERE upper(lab_test_name) LIKE '%SARS-COV-2%'  
OR upper(lab_test_name) LIKE '%COVID%'
```

The key idea is to transform both the column value and the search pattern into the same case before matching.

Joining With LIKE

LIKE can also be used in a join condition when two tables need to be matched by a pattern rather than exact equality.

- standard join logic expects exact matches
- standard pattern:

```
FROM table1  
JOIN table2  
ON table1.foo = table2.bar
```

	• LIKE join logic allows one value to contain another value as part of a larger string • example idea: ○ brown cow can match cow ○ striped cat can match cat ○ large elephant can match elephant • the join condition must build a wildcard pattern around the value being matched • PostgreSQL example:<pre>FROM table1 JOIN table2 ON table1.foo LIKE '%' table2.bar '%'</pre> • the wildcard symbols are concatenated onto the value from table2.bar The resulting pattern checks whether table1.foo contains the value from table2.bar.
Casting Before Pattern Matching	LIKE requires string-compatible values. • if the value being matched is not stored as a string, it may need to be cast to a string type first • casting allows the value to participate in concatenation and pattern matching • this is especially relevant when matching numeric or coded values stored in non-text columns

Performance Caution	<i>LIKE</i> joins can be slow. • pattern matching is more expensive than exact matching • joining with wildcard patterns can become especially slow on large tables • this approach is usually not ideal for clean or well-structured data • it may still be useful when messy data prevents exact joins • <i>LIKE</i> joins should be treated as a practical fallback rather than the preferred design
MariaDB Note	The filtering examples work similarly in MariaDB. • MariaDB example: <pre>SELECT * FROM patient_labs WHERE LOWER (lab_test_name) LIKE '%sars-cov-2%' OR LOWER (lab_test_name) LIKE '%covid%'</pre> • the PostgreSQL join example uses for concatenation • MariaDB normally uses CONCAT() instead • MariaDB example: <pre>FROM table1 JOIN table2 ON table1.foo LIKE CONCAT ('%', table2.bar, '%')</pre> • if table2.bar is not already text, convert it before using it in the pattern
Key Ideas	• <i>LIKE</i> supports inexact text matching • % matches any sequence of characters • _ matches one character

	• <i>LIKE</i> in <i>WHERE</i> filters rows based on a pattern • <i>LOWER()</i> or <i>UPPER()</i> can reduce case-related matching problems • <i>LIKE</i> can be used in join conditions for fuzzy matching • <i>wildcards</i> can be concatenated around column values to build dynamic patterns • <i>LIKE joins</i> can be slow and should be used carefully with large tables • when possible, <i>clean and standardize data</i> before relying on fuzzy pattern joins
Regular Expressions	
Purpose Of Regular Expressions	Regular expressions (regex) are used for pattern matching when exact matching is not flexible enough. • can identify whether a value follows a particular structure • useful when a column contains messy, inconsistent, or mixed-format data • allows a query to search for patterns rather than exact values
Why Pattern Matching Matters	Some data cannot be reliably queried with simple equality comparisons. • example problem: ○ identify which values in a messy text column could be United States phone numbers • some values may look valid • some values may be too long • some values may be too short • some values may contain letters or invalid characters

	• a human can visually inspect a few rows • but this does not scale to millions or billions of records SQL needs a formal way to express the idea of looks like a phone number.
Regex As A Boolean Test	A regular expression can be used in a WHERE clause to test whether each value matches a pattern. • the database evaluates each row against the regex pattern • if the value matches the pattern, the test returns true • if the value does not match the pattern, the test returns false • the query keeps only the rows where the regex test is true This makes regex useful for filtering messy data based on structure.
Phone Number Pattern Example	A United States phone number has a recognizable structure, but it may appear in several formats. • core structure: ○ three-digit area code ○ three-digit prefix ○ four-digit line number • possible optional elements: ○ country code such as +1 ○ parentheses around the area code ○ hyphens between number groups ○ dots between number groups

- spaces between number groups
- no separators at all

Regex Pattern Matching

Use regular expressions to identify values that match a phone number format.

Find the right structure.

Example Input
Real-world data can be messy.

- 704-555-1234
- (704) 555-1234
- 704.555.1234
- 704 555 1234
- +1 704-555-1234
- 7045551234
- 555-1234
- abc-555-1234
- (704)555-12
- 704-555-12345

REGEX PATTERN

`^(\(?d{3}\)?[-.\s]?d{3}[-.\s]?d{4})$`

Matches common U.S. phone number formats with optional parentheses, spaces, or dashes.

PATTERN MATCH
NO MATCH

Validation Results

Valid (matches pattern)

- 704-555-1234 ✓
- (704) 555-1234 ✓
- 704.555.1234 ✓
- 704 555 1234 ✓
- +1 704-555-1234 ✓
- 7045551234 ✓

Invalid (does not match)

- 555-1234 ✗
- abc-555-1234 ✗
- (704)555-12 ✗
- 704-555-12345 ✗

Pattern Breakdown (key elements)

Symbol	Description
^	start of string
\(\)?	optional parentheses
d{3}	area code (3 digits)
[-.\s]?	optional separator (-, ., or space)
d{3}	prefix (3 digits)
[-.\s]?	optional separator (-, ., or space)
d{4}	line number (4 digits)
\$	end of string

CLEAN DATA BUILDS TRUST.

Regular expressions let you teach SQL what a valid phone number looks like so you can find structure in messy data.

QUERIES
PATTERNS
INSIGHTS

ChatGPT 5.6 Sol

A useful phone-number regex must allow expected variation without becoming so loose that it matches invalid data.

Balancing Flexibility And Precision

Regex design often requires balancing what should match against what should be rejected.

- a strict pattern may miss valid values written in an unexpected format
- a loose pattern may capture values that are not actually valid
- a phone-number regex can match most United States phone numbers
- a regex that handles every international phone-number format would be much more complex
- the pattern should be designed around the data and the purpose of the query

Basic Regex Concepts

Regex syntax uses special symbols to describe patterns.

- `\d` matches any digit
- `|D` matches any non-digit
- word-related patterns can match letters, numbers, or word boundaries
- space-related patterns can match whitespace
- quantifiers can describe how many times something should appear
- optional elements can allow a pattern to match values with or without certain characters

Regex can represent almost any character or text structure that can appear in computer-readable text.

Learning Regex

Regex syntax can look dense at first, but it is learned best through practice.

Regular Expressions (Regex)
A powerful way to find patterns in text.

Regex lets us match complex patterns with simple rules.

Regex Symbol	Meaning	Example
.	matches any character	c.t → cat, cot, cut
\d	matches any digit (0-9)	\d\d\d → 123
\D	matches any non-digit	\D\D → AB, #1
\w	matches any word character (a-z, A-Z, 0-9, _)	\w+ → user_1
\W	matches any non-word character	\W → -, ?, !
\s	matches any white space (space, \t, \n, \r, \f)	\s+ → space or tab
\S	matches any non-white space	\S+ → text
[abc]	matches any character in brackets	[aeiou] → a, e, i, o, u
^	matches start of string	^A → Apple, Async
\$	matches end of string	\.txt\$ → file.txt
*	matches zero or more of previous	ab* → a, ab, abb
+	matches one or more of previous	ab+ → ab, abb, abbb
?	matches zero or one of previous	colou?r → color, colour
{n}	matches exactly n times	\d{3} → 123
{n,}	matches at least n times	\d{2,} → 12, 123, 1234
{n,m}	matches between n and m times	\d{2,4} → 12, 123, 1234

Patterns unlock data.

Example Pattern
U.S. phone number (with optional formats)
`^(?(\d{3})\)?[-.]?\d{3}[-.]?\d{4})$`

Matches:

- ✓ 704-555-1234
- ✓ (704) 555-1234
- ✓ 704.555.1234
- ✓ 704 555 1234

Does not match:

- ✗ 555-1234
- ✗ abc-555-1234
- ✗ 704-555-12345
- ✗ (704)555-12

Same data. Many formats. One pattern.

EXPLORE
LEARN
APPLY

ChatGPT 5.6 Sol

	• regex expressions often look confusing before the symbols are understood • cheat sheets are useful references while building patterns • regex is usually learned by experimentation rather than memorization Building small patterns and testing them against example strings is the most practical way to learn.
Testing And Debugging Regex	Online regex editors help construct, test, and debug patterns. • regex101 is one example of an online regex testing tool • test strings can be entered into the editor • the regex pattern highlights values that match • non-highlighted values reveal cases the pattern did not capture • the editor can explain pieces of the regex syntax This makes it easier to understand why a pattern does or does not match.
Password Validation Example	Regex can validate whether a string satisfies several structural rules. • example password rules: ○ at least one capital letter ○ at least one lowercase letter ○ at least one number ○ at least one symbol ○ a weak password such as password1 may fail because it does not satisfy all rules

	○ a stronger password may match when it contains all required character types This demonstrates how regex can test multiple pattern requirements at once.
Why Regex Is Needed In SQL	SQL returns exactly what the query asks for. • exact equality only finds values that match exactly • messy data often requires fuzzier matching • regex provides a way to describe that fuzziness precisely Regex is especially useful when the desired data shares a structure but not an exact value.
Common Use Cases	Regex is useful when data must be identified by pattern rather than exact content. • phone-number detection • email-format validation • password-format validation • searching messy text fields • finding values that look like lab results • finding date-like values inside paragraphs Identifying structured information inside unstructured or semi-structured text,

Regex In Text Columns	Databases can store long text values such as sentences or paragraphs. • regex can search those text values for embedded structures • example: ○ find paragraphs that contain any date-like value • dates can appear in many formats with different punctuation • a regex can describe those possible formats and return matching text This makes regex useful for extracting structured signals from larger text fields.
Key Ideas	• regular expressions support pattern matching • regex is useful when equality matching is too rigid • a regex test can return true or false for each row • regex can filter messy data by structure • regex patterns must balance flexibility and precision • complex patterns should be tested against realistic examples • online editors can help build and debug regex expressions • regex is a powerful skill for querying, validating, and cleaning database text
MariaDB Note	MariaDB supports regular-expression matching, but the syntax may differ from PostgreSQL. • common MariaDB operators: ○ <i>REGEXP</i> ○ <i>RLIKE</i>

	• example concept: ○ <code>column_name REGEXP 'pattern'</code> • MariaDB regex behavior depends on the version and regular-expression engine • patterns should be tested directly in MariaDB before relying on them for cleaning or validation • when translating from PostgreSQL examples, verify both the matching operator and the regex syntax before running the query
Using Regular Expressions in SQL	
Regex As Flexible Pattern Matching	Regular expressions provide a more flexible form of pattern matching than LIKE. • <i>LIKE</i> supports simple pattern matching with wildcards • regex supports more complex and sophisticated patterns • regex is useful when the target pattern cannot be described clearly with only % and _ Regex can identify patterns inside strings, count pattern matches, test whether a pattern exists, or replace matching text.
Database-Specific Syntax	Regex syntax and function support vary by relational database management system. • the exact regex functions depend on the RDBMS • the examples in this material use PostgreSQL • when moving between databases, regex syntax and available functions should be checked before reuse

**PostgreSQL
Regex Functions**

PostgreSQL provides several functions that can use regular expressions.

- `substring(string from pattern)`
 - extracts a substring that matches the regex pattern
 - returns the matching substring when a match exists
 - useful when the goal is to pull out the matching portion of a larger string
- `regexp_count(string, pattern)`
 - counts the number of times a regex pattern appears within a string
 - useful when the goal is to measure how often a pattern occurs
 - can support text analysis, validation, or data-quality checks
- `regexp_like(string, pattern)`
 - returns true or false depending on whether the regex pattern is found in the string
 - similar in purpose to *LIKE*
 - more flexible than *LIKE* because the pattern is defined with regex rather than simple wildcards
 - useful in *WHERE* clauses when filtering rows by complex text patterns
- `regexp_replace(source, pattern, replacement)`
 - replaces text that matches a regex pattern with a specified replacement string
 - useful for cleaning or standardizing text

	○ can transform strings when the values to be replaced follow a pattern rather than one exact value
Regex Documentation	PostgreSQL documentation provides more detail on regex functions and syntax. • the documentation includes additional regex functions • the documentation includes usage examples • PostgreSQL also provides a regex syntax reference or cheat sheet • regex cheat sheets are useful because regex patterns can become dense and difficult to memorize
Key Ideas	• regex extends pattern matching beyond the simpler LIKE expression • <i>LIKE</i> is useful for basic wildcard matching • regex is useful for complex text patterns • PostgreSQL supports regex functions for extracting, counting, testing, and replacing pattern matches • regex syntax is database-specific • documentation and cheat sheets are important references when writing or debugging regex patterns